

LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 14 – SORTING



NAMA : YAAFI DZAKWAN

NIM : 109082500173

KELAS S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Materi ini membahas dua metode pengurutan data mendasar, yaitu *Selection Sort* (pengurutan secara seleksi) dan *Insertion Sort* (pengurutan secara sisip). **Selection Sort** bekerja dengan cara memindai seluruh array yang tersisa untuk mencari nilai ekstrim, baik berupa nilai terkecil maupun terbesar. Setelah nilai tersebut ditemukan melalui pencarian indeks, data akan langsung ditukar tempatnya dengan elemen yang berada di posisi paling kiri pada rentang data tersisa. Proses pencarian dan penukaran posisi ini diulangi terus-menerus hingga seluruh data bergeser ke tempat yang seharusnya secara berurutan.

Sementara itu, **Insertion Sort** menggunakan pendekatan yang berbeda karena tidak mencari nilai ekstrim di awal proses. Algoritma ini langsung mengambil satu elemen dari kelompok data yang belum terurut, lalu mencari posisi yang tepat untuk elemen tersebut di dalam kelompok data yang sudah terurut menggunakan pencarian sekuensial. Saat posisinya ditemukan, program akan menggeser elemen-elemen lain yang lebih besar/kecil ke arah kanan untuk menciptakan ruang kosong, kemudian menyisipkan nilai tersebut ke dalamnya.

B. Guided

1. Guided 1

```
package main

import "fmt"

func selectionSortArray(angka *[5]int){
    var idx_min, i, j int
    for i = 0; i < len(angka) - 1; i++ { //loop luar
        (iterasi sortingnya)
        idx_min = i
        for j = i + 1; j < len(angka); j++ { //loop dalam
            (mencari nilai ekstrim)
            if angka[j] <= angka[idx_min] { //sorting
                terkecil ke terbesar
                idx_min = j
            }
        }
        if idx_min != i {
            angka[i], angka[idx_min] = angka[idx_min],
            angka[i]
        }
    }
}
```

```

func main(){
    var arrAngka [5]int

    for i := 0; i < len(arrAngka); i++ {
        fmt.Printf("Masukkan data angka ke-%d : ", i)
        fmt.Scan(&arrAngka[i])
    }
    fmt.Println()

    fmt.Println("=== SEBELUM DISORTING ===")
    for i := 0; i < len(arrAngka); i++ {
        fmt.Print(arrAngka[i], " - ")
    }

    fmt.Println("\n=== SETELAH DISORITNG ===")
    seletionSortArray(&arrAngka)
    for i := 0; i < len(arrAngka); i++ {
        fmt.Print(arrAngka[i], " - ")
    }
}

```

Output :

```

PS C:\Users\YAAFI\OneDrive\文件\109082500173_Yaafi-Dzakwan> go run
173_Yaafi-Dzakwan\Week 12-Modul 14\Guided\Guided1\n01.go"
Masukkan data angka ke-0 : 15
Masukkan data angka ke-1 : 10
Masukkan data angka ke-2 : 5
Masukkan data angka ke-3 : 8
Masukkan data angka ke-4 : 2

=== SEBELUM DISORTING ===
15 - 10 - 5 - 8 - 2 -
=== SETELAH DISORITNG ===
2 - 5 - 8 - 10 - 15 -

```

2. Guided 2

```

package main

import "fmt"

type mahasiswa struct {
    nama, nim string
    IPK       float64
}

```

```

func SelectionSortArray(sMHS *[5]mahasiswa) {
    var idx_min, i, j int
    for i = 0; i < len(sMHS)-1; i++ { //loop luar
        idx_min = i
        for j = i + 1; j < len(sMHS); j++ { //loop dalam
            mencari nilai ekstrim
            if sMHS[j].IPK < sMHS[idx_min].IPK { //sorting
                terkecil ke terbesar
                idx_min = j
            }
        }
        if idx_min != i {
            sMHS[i], sMHS[idx_min] = sMHS[idx_min], sMHS[i]
        }
    }
}

func main() {
    var structMHS [5]mahasiswa

    for i := 0; i < len(structMHS); i++ {
        fmt.Printf("--- Masukkan Data Mahasiswa Ke-%d ---\n", i)
        fmt.Print("Nama : ")
        fmt.Scan(&structMHS[i].nama)
        fmt.Print("NIM : ")
        fmt.Scan(&structMHS[i].nim)
        fmt.Print("IPK : ")
        fmt.Scan(&structMHS[i].IPK)
    }
    fmt.Println()

    fmt.Println(" === SEBELUM SORITNG === ")
    for i := 0; i < len(structMHS); i++ {
        fmt.Printf("--- Data Mahasiswa Ke-%d ---\n", i)
        fmt.Printf("Nama : %s\n", structMHS[i].nama)
        fmt.Printf("NIM : %s\n", structMHS[i].nim)
        fmt.Printf("IPK : %.2f\n", structMHS[i].IPK)
    }
    fmt.Println("=====")
    fmt.Println()

    SelectionSortArray(&structMHS)
    fmt.Println(" === SETELAH SORITNG === ")
    for i := 0; i < len(structMHS); i++ {

```

```

        fmt.Printf("--- Data Mahasiswa Ke-%d ---\n", i)
        fmt.Printf("Nama : %s\n", structMHS[i].nama)
        fmt.Printf("NIM : %s\n", structMHS[i].nim)
        fmt.Printf("IPK : %.2f\n", structMHS[i].IPK)
    }
    fmt.Println("=====")
    fmt.Println()
}

```

Output :

```

PS C:\Users\YAAFI\OneDrive\文件\109082500173_Yaafi-Dzakwan> go run
173_Yaafi-Dzakwan\Week 12-Modul 14\Guided\Guided2\n02.go"
--- Masukkan Data Mahasiswa Ke-0 ---
Nama : Yaafi
NIM : 109082500173
IPK : 3.56
--- Masukkan Data Mahasiswa Ke-1 ---
Nama : Danis
NIM : 109082500172
IPK : 3.66
--- Masukkan Data Mahasiswa Ke-2 ---
Nama : Alif
NIM : 10908250003
IPK : 3.77
--- Masukkan Data Mahasiswa Ke-3 ---
Nama : Tian
NIM : 109082500673
IPK : 3.78
--- Masukkan Data Mahasiswa Ke-4 ---
Nama : sigit
NIM : 229082500123
IPK : 3,55

=== SEBELUM SORITNG ===
--- Data Mahasiswa Ke-0 ---
Nama : Yaafi
NIM : 109082500173
IPK : 3.56

```

3. Guided3

```

package main

import "fmt"

const nMax int = 4321

type arrInt [nMax]int

func insertionSort(T *arrInt, n int) {
    /* I.S. terdefinisi array T yang berisi n bilangan bulat

```

```

        F.S. array T terurut secara mengecil atau descending dengan
        INSERTION SORT */

        var temp, i, j int

        i = 1
        for i <= n-1 {
            j = i
            temp = T[j]

            for j > 0 && temp > T[j-1] {
                T[j] = T[j-1]
                j = j - 1
            }

            T[j] = temp
            i = i + 1
        }
    }

func main() {
    var data arrInt
    var n int

    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        fmt.Scan(&data[i])
    }

    insertionSort(&data, n)

    for i := 0; i < n; i++ {
        fmt.Print(data[i], " ")
    }
}

```

Output :

```

PS C:\Users\YAAFI\OneDrive\文件\109082500173_Yaafi-Dzakwan> go run
173_Yaafi-Dzakwan\Week 12-Modul 14\Guided\Guided3\no3.go
2
1 3 7 5 2
3 1

```

4. Guided 4

```
package main

import "fmt"

const nMax int = 100

type arrString [nMax]string

func insertionSortString(T *arrString, n int) {
    var temp string
    var i, j int

    i = 1
    for i <= n-1 {
        j = i
        temp = T[j]

        for j > 0 && temp < T[j-1] {
            T[j] = T[j-1]
            j = j - 1
        }

        T[j] = temp
        i = i + 1
    }
}

func main() {
    var data arrString
    var n int

    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        fmt.Scan(&data[i])
    }

    insertionSortString(&data, n)
```

```

    for i := 0; i < n; i++ {
        fmt.Print(data[i], " ")
    }
}

```

Output :

```

PS C:\Users\YAAFI\OneDrive\文件\109082500173_Yaafi-Dzakwan> go run
173_Yaafi-Dzakwan\Week 12-Modul 14\Guided\Guided4\n04.go"
5
Budi Andi Caca Deni Eko
Andi Budi Caca Deni Eko

```

C. Unguided

1. Soal Unguided 1

Hercules, preman terkenal seantero ibukota, memiliki kerabat di banyak daerah. Tentunya Hercules sangat suka mengunjungi semua kerabatnya itu. Diberikan masukan nomor rumah dari semua kerabatnya di suatu daerah, buatlah program rumahkerabat yang akan menyusun nomor-nomor rumah kerabatnya secara terurut membesar menggunakan algoritma selection sort.

Jawaban :

```

package main

import "fmt"

// Fungsi Selection Sort untuk mengurutkan secara menaik
(Ascending)
func selectionSort(arr []int) {
    n := len(arr)
    for i := 0; i < n-1; i++ {
        minIdx := i
        for j := i + 1; j < n; j++ {
            if arr[j] < arr[minIdx] {
                minIdx = j
            }
        }
        if minIdx != i {
            arr[i], arr[minIdx] = arr[minIdx], arr[i]
        }
    }
}

func main() {

```



```

var n int
// Membaca jumlah daerah
fmt.Scan(&n)

for i := 0; i < n; i++ {
    var m int
    // Membaca jumlah rumah di daerah tersebut
    fmt.Scan(&m)

    rumah := make([]int, m)
    for j := 0; j < m; j++ {
        fmt.Scan(&rumah[j])
    }

    // Urutkan data
    selectionSort(rumah)

    // Cetak hasil sesuai format
    for j := 0; j < m; j++ {
        fmt.Print(rumah[j])
        if j < m-1 {
            fmt.Print(" ")
        }
    }
    fmt.Println()
}
}

```

Output :

```

PS C:\Users\YAAFI\OneDrive\文件\109082500173_Yaafi-Dzakwan> go run
173_Yaafi-Dzakwan\Week 12-Modul 14\Unguided\Unguided1\n01.go"
3
5 2 1 7 9 13
1 2 7 9 13
6 189 15 27 39 75 133
15 27 39 75 133 189
3 4 9 1
1 4 9

```

Penjelasan :

Selection Sort adalah metode pengurutan data yang bekerja dengan cara memindai seluruh elemen di dalam array untuk mencari nilai yang paling kecil, lalu menukar nilai terkecil tersebut dengan elemen yang berada di posisi baris depan yang belum terurut. Proses pencarian dan penukaran ini terus diulangi untuk posisi kedua, ketiga, dan seterusnya hingga seluruh data bergeser ke posisi yang tepat dan menghasilkan urutan yang rapi dari kecil ke besar.

2. Soal Unguided 2

Belakangan diketahui ternyata Hercules itu tidak berani menyeberang jalan, maka selalu diusahakan agar hanya menyeberang jalan sesedikit mungkin, hanya diujung jalan. Karena nomor rumah sisi kiri jalan selalu ganjil dan sisi kanan jalan selalu genap, maka buatlah program kerabat dekat yang akan menampilkan nomor rumah mulai dari nomor yang ganjil lebih dulu terurut membesar dan kemudian menampilkan nomor rumah dengan nomor genap terurut mengecil.

Jawaban :

```
package main

import "fmt"

// Fungsi Selection Sort untuk mengurutkan secara menaik
(Ascending)
func selectionSortAscending(arr []int) {
    n := len(arr)
    for i := 0; i < n-1; i++ {
        minIdx := i
        for j := i + 1; j < n; j++ {
            if arr[j] < arr[minIdx] {
                minIdx = j
            }
        }
        if minIdx != i {
            arr[i], arr[minIdx] = arr[minIdx], arr[i]
        }
    }
}

// Fungsi Selection Sort untuk mengurutkan secara menurun
(Descending)
func selectionSortDescending(arr []int) {
    n := len(arr)
    for i := 0; i < n-1; i++ {
        maxIdx := i
        for j := i + 1; j < n; j++ {
            if arr[j] > arr[maxIdx] {
                maxIdx = j
            }
        }
        if maxIdx != i {
            arr[i], arr[maxIdx] = arr[maxIdx], arr[i]
        }
    }
}
```

```

func main() {
    var n int
    // Membaca jumlah daerah
    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        var m int
        // Membaca jumlah rumah di daerah tersebut
        fmt.Scan(&m)

        var ganjil []int
        var genap []int

        for j := 0; j < m; j++ {
            var nomor int
            fmt.Scan(&nomor)
            // Pisahkan ganjil dan genap
            if nomor%2 != 0 {
                ganjil = append(ganjil, nomor)
            } else {
                genap = append(genap, nomor)
            }
        }

        // Urutkan masing-masing kelompok slice
        selectionSortAscending(ganjil)
        selectionSortDescending(genap)

        // Gabungkan saat pencetakan data
        first := true
        for _, num := range ganjil {
            if !first {
                fmt.Print(" ")
            }
            fmt.Print(num)
            first = false
        }
        for _, num := range genap {
            if !first {
                fmt.Print(" ")
            }
            fmt.Print(num)
            first = false
        }
        fmt.Println()
    }
}

```

Output :

```
PS C:\Users\YAAFI\OneDrive\文件\109082500173_Yaafi-Dzakwan> go run
173_Yaafi-Dzakwan\Week 12-Modul 14\Unguided\Unguided2\no2.go"
3
5 2 1 7 9 13
1 7 9 13 2
6 189 15 27 39 75 133
15 27 39 75 133 189
3 4 9 1
1 9 4
```

Penjelasan :

Materi ini berfokus pada pemisahan dan pengurutan data menggunakan dua aturan arah yang berbeda berdasarkan sifat bilangan tersebut. Sejak awal data dibaca, angka ganjil dan angka genap langsung dipisahkan ke dalam dua kelompok yang berbeda, di mana kelompok ganjil kemudian diurutkan secara menaik dari nilai terkecil ke terbesar, sedangkan kelompok genap diurutkan secara menurun dari nilai terbesar ke terkecil. Hasil akhirnya ditampilkan dengan cara menyambung barisan angka ganjil yang sudah terurut tadi langsung dengan barisan angka genap di belakangnya dalam satu baris keluaran yang sama.

3. Soal Unguided 3

Diberikan n data integer positif dalam keadaan terurut membesar dan sebuah integer lain k, apakah bilangan k tersebut ada dalam daftar bilangan yang diberikan? Jika ya, berikan indeksnya, jika tidak sebutkan "TIDAK ADA".

Jawaban :

```
package main

import "fmt"

// Fungsi Insertion Sort untuk mengurutkan secara menaik
(Ascending)
func insertionSort(arr []int) {
    n := len(arr)
    for i := 1; i < n; i++ {
        key := arr[i]
        j := i - 1

        // Geser elemen-elemen yang lebih besar dari key
        ke kanan
        for j >= 0 && arr[j] > key {
            arr[j+1] = arr[j]
            j--
        }
        arr[j+1] = key
    }
}
```

```

    }
}

func main() {
    var num int
    var data []int

    // Loop untuk membaca input sampai ditemukan bilangan
    negatif
    for {
        fmt.Scan(&num)
        if num < 0 {
            break
        }
        data = append(data, num)
    }

    // Jika input kosong, program langsung selesai
    if len(data) == 0 {
        return
    }

    // 1. Urutkan data menggunakan Insertion Sort
    insertionSort(data)

    // 2. Cetak baris pertama: isi array yang sudah terurut
    for i := 0; i < len(data); i++ {
        fmt.Print(data[i])
        if i < len(data)-1 {
            fmt.Print(" ")
        }
    }
    fmt.Println()

    // 3. Periksa jarak antar elemen
    // Jika elemen kurang dari atau sama dengan 1, otomatis
    dianggap berjarak tetap 0 atau tidak punya selisih
    if len(data) <= 1 {
        fmt.Println("Data berjarak 0")
        return
    }

    // Ambil selisih pertama sebagai acuan awal
    selisihAwal := data[1] - data[0]
    isTetap := true

    // Periksa seluruh sisa elemen bersebelahan
    for i := 1; i < len(data)-1; i++ {
        if data[i+1]-data[i] != selisihAwal {
            isTetap = false
            break
        }
    }
}

```

```

    }
}

// 4. Cetak baris kedua: status jarak
if isTetap {
    fmt.Printf("Data berjarak %d\n", selisihAwal)
} else {
    fmt.Println("data berjarak tidak tetap")
}
}

```

Output :

```

PS C:\Users\YAAFI\OneDrive\文件\109082500173_Yaafi-Dzakwan> go run
173_Yaafi-Dzakwan\Week 12-Modul 14\Unguided\Unguided3\no3.go
31 13 25 43 1 7 19 37 -5
1 7 13 19 25 31 37 43
Data berjarak 6

```

Penjelasan :

Program ini membaca sekumpulan bilangan bulat positif yang dimasukkan pengguna secara dinamis dan akan berhenti otomatis ketika mendeteksi bilangan negatif. Setelah data terkumpul di dalam *slice*, angka-angka tersebut diurutkan dari yang terkecil ke terbesar menggunakan algoritma **Insertion Sort**, yang bekerja dengan cara menyisipkan setiap elemen ke posisi yang tepat di area yang sudah terurut. Di tahap akhir, program membandingkan selisih antara setiap angka yang bersebelahan; jika semua angka memiliki selisih (jarak) yang sama konstan, program akan mencetak jarak tersebut (misalnya: "Data berjarak 6"), tetapi jika ditemukan ada selisih yang berbeda, program akan menampilkan status "data berjarak tidak tetap".

4. Soal Unguided 4

Sebuah program perpustakaan digunakan untuk mengelola data buku di dalam suatu perpustakaan. Misalnya terdefinisi struct dan array seperti berikut ini: Masukan terdiri dari beberapa baris. Baris pertama adalah bilangan bulat *N* yang menyatakan banyaknya data buku yang ada di dalam perpustakaan. *N* baris berikutnya, masing-masingnya adalah data buku sesuai dengan atribut atau field pada struct. Baris terakhir adalah bilangan bulat yang menyatakan rating buku yang akan dicari.

Jawaban :

```
package main

import "fmt"

// Batas maksimum array sesuai spesifikasi soal
const nMax = 7919

// Definisi Struct Buku
type Buku struct {
    id, judul, penulis, penerbit string
    eksemplar, tahun, rating      int
}

// Definisi tipe data tipe array DaftarBuku
type DaftarBuku [nMax]Buku

// 1. Prosedur DaftarkanBuku (Membaca data dari input)
func DaftarkanBuku(pustaka *DaftarBuku, n int) {
    for i := 0; i < n; i++ {
        fmt.Scan(&pustaka[i].id, &pustaka[i].judul,
            &pustaka[i].penulis, &pustaka[i].penerbit)
        fmt.Scan(&pustaka[i].eksemplar, &pustaka[i].tahun,
            &pustaka[i].rating)
    }
}

// 2. Prosedur UrutBuku (Insertion Sort - Descending
// berdasarkan rating)
func UrutBuku(pustaka *DaftarBuku, n int) {
    for i := 1; i < n; i++ {
        key := pustaka[i]
        j := i - 1

        // Geser elemen yang ratingnya lebih kecil dari
        // key.rating ke kanan
        for j >= 0 && pustaka[j].rating < key.rating {
            pustaka[j+1] = pustaka[j]
            j--
        }
        pustaka[j+1] = key
    }
}

// 3. Prosedur CetakTerfavorit (Mencari rating tertinggi
// secara manual dari array acak)
func CetakTerfavorit(pustaka DaftarBuku, n int) {
    if n <= 0 {
        return
    }
}
```

```

maxRating := pustaka[0].rating
idxMax := 0
for i := 1; i < n; i++ {
    if pustaka[i].rating > maxRating {
        maxRating = pustaka[i].rating
        idxMax = i
    }
}

b := pustaka[idxMax]
fmt.Printf("%s %s %s %d\n", b.judul, b.penulis,
b.penerbit, b.tahun)
}

// 4. Prosedur Cetak5Terbaru (Mencetak hingga 5 judul buku
dengan rating tertinggi)
func Cetak5Terbaru(pustaka DaftarBuku, n int) {
    limit := 5
    if n < 5 {
        limit = n
    }

    for i := 0; i < limit; i++ {
        fmt.Printf("%s\n", pustaka[i].judul)
    }
}

// 5. Prosedur CariBuku (Binary Search pada array yang
terurut menurun)
func CariBuku(pustaka DaftarBuku, n int, r int) {
    left := 0
    right := n - 1
    foundIdx := -1

    for left <= right {
        mid := (left + right) / 2
        if pustaka[mid].rating == r {
            foundIdx = mid
            break
        } else if pustaka[mid].rating < r {
            right = mid - 1 // Cari ke kiri karena array
terurut menurun
        } else {
            left = mid + 1 // Cari ke kanan
        }
    }

    if foundIdx != -1 {
        b := pustaka[foundIdx]
    }
}

```



```

        fmt.Printf("%s %s %s %d %d %d\n", b.judul,
b.penulis, b.penerbit, b.tahun, b.eksemplar, b.rating)
    } else {
        fmt.Println("Tidak ada buku dengan rating seperti
itu")
    }
}

// Fungsi Utama: Mengatur alur eksekusi sesuai instruksi
soal
func main() {
    var n int
    fmt.Scan(&n)

    var pustaka DaftarBuku

    // Membaca seluruh data buku
    DaftarkanBuku(&pustaka, n)

    // Membaca rating yang akan dicari pada baris terakhir
input
    var ratingDicari int
    fmt.Scan(&ratingDicari)

    // --- PROSES OUTPUT ---

    // Baris 1 Output: Cetak buku terfavorit saat kondisi
array belum terurut
    CetakTerfavorit(pustaka, n)

    // Proses pengurutan internal menggunakan Insertion Sort
secara Descending
    UrutBuku(&pustaka, n)

    // Baris 2 Output: Cetak 5 besar judul buku rating
tertinggi
    Cetak5Terbaru(pustaka, n)

    // Baris 3 Output: Pencarian biner berdasarkan rating
yang diinput di akhir
    CariBuku(pustaka, n, ratingDicari)
}

```

Output :

```

173_Yaafi-Dzakwan\Week 12-Modul 14\Unguided\Unguided4\n04.go"
6
B01 Laskar_Pelangir Andrea_Hirata Bentang 10 2005 92
B02 Bumi_Manusia Pramoedya_Ananta Hasta_Mitra 5 1980 98
B03 Cantik Itu Luka Eka_Kurniawan Gramedia 8 2002 85
B04 Dilan_1990 Pidi_Baiq Pastel_Books 15 2014 88
B05 Supernova Dee_Lestari Truedee 12 2001 90
B06 Hujan_Bulan_Juni Sapardi_Djoko Grasindo 7 1994 95
90
Bumi_Manusia Pramoedya_Ananta Hasta_Mitra 1980
Bumi_Manusia
Hujan_Bulan_Juni
Laskar_Pelangir
Supernova
Dilan_1990
Supernova Dee_Lestari Truedee 2001 12 90

```

Penjelasan :

Program ini bekerja secara modular dimulai dengan membaca N data buku, lalu mencari dan mencetak buku terfavorit (rating tertinggi) saat data masih acak. Selanjutnya, seluruh data buku diurutkan secara menurun (*descending*) berdasarkan ratingnya menggunakan metode **Insertion Sort**. Setelah terurut, program akan menampilkan 5 judul buku dengan rating tertinggi di baris teratas, dan terakhir melakukan pencarian cepat menggunakan **Binary Search** untuk menemukan buku dengan spesifikasi rating yang dicari oleh pengguna.

D. Kesimpulan

Berdasarkan pembahasan materi di atas, dapat disimpulkan bahwa *Selection Sort* dan *Insertion Sort* merupakan dua metode pengurutan data yang efektif dengan karakteristik serta mekanismenya masing-masing, di mana *Selection Sort* mengandalkan pencarian nilai ekstrim dan pertukaran tempat, sedangkan *Insertion Sort* mengandalkan pencarian sekuensial posisi dan penyisipan elemen. Selain itu, kedua algoritma ini terbukti sangat adaptif karena tidak hanya bisa digunakan untuk mengurutkan angka acak secara membesar (*ascending*) maupun mengecil (*descending*), tetapi juga mampu mengorganisasi struktur data yang kompleks seperti *struct* dengan cara menyesuaikan kriteria *field* yang ingin dijadikan sebagai dasar acuan pengurutan.